

Question Bank — Week 14

Graphs and Graph Searches (Lecture 14) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	1
Part B — Short answer and essay	3
Part C — Trace the algorithms	4
Part D — Graph state	7
Part E — Complexity analysis	8
Part F — Find and fix the bug	8
Part G — Write the code	10

Questions: [week14-questions.md](#). Commit to your own answer before reading one here. Every trace and output below was produced by running the reference solution, `solutions/dsa/graph.py`.

For reference, the neighbour lists of **H** are $A: B D \cdot B: A C E \cdot D: A E \cdot C: B F \cdot E: B D F \cdot F: C E G \cdot G: F$, and `H.nodes()` is `A B D C E F G` (the order the nodes were first added).

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	c	M07	b	M13	b	M19	d
M02	a	M08	d	M14	c	M20	a
M03	d	M09	a	M15	d	M21	c
M04	a	M10	b	M16	c	M22	d
M05	b	M11	b	M17	a		
M06	c	M12	a	M18	c		

W14-M01 — c. A $V \times V$ grid, whatever E is. $O(V + E)$ (a) is the adjacency list.

W14-M02 — a. First discovered, first expanded. A stack (b) gives DFS; a priority queue (c) gives Dijkstra's algorithm, beyond this course.

W14-M03 — d. The degrees add up to $2 + 3 + 2 + 2 + 3 + 3 + 1 = 16$, and every edge is counted at both ends: $E = 16/2 = 8$. 16 (a) forgets to halve. (These are the degrees of H.)

W14-M04 — a. The queue holds all of distance d before any of distance $d + 1$, so a vertex is discovered while BFS is expanding the layer just before it. It never tries every path (c) — that would be exponential. Marking on dequeue (b) is a bug (W14-B1).

W14-M05 — b. A adds B and D; B adds C and E; D adds nothing new; C adds F; F adds G (W14-T1). (a) is alphabetical order, which BFS does not use.

W14-M06 — c. $A \rightarrow B \rightarrow C \rightarrow F \rightarrow E \rightarrow D$, back up to F, then G (W14-T2). (a) is what you get if you think of BFS layers; (b) goes from C to E, but C and E are not adjacent.

W14-M07 — b. D's list is A, E, so A is taken first and becomes B's parent; B then discovers C. D-E-B-C (c) and D-E-F-C (a) are also three edges long — equally short — but BFS keeps the first parent it records (W14-T4).

W14-M08 — d. One cell, `matrix[i][j]`, after two $O(1)$ hash-map lookups for i and j . $O(\deg u)$ (b) is the list.

W14-M09 — a. The whole row of V cells must be scanned to find the 1s. That is the matrix's price.

W14-M10 — b. V vertices, each costing a V -cell row scan: V^2 . The number of edges does not change the work.

W14-M11 — b. Each undirected edge $u-v$ is stored in u 's list and in v 's list. The same fact is $\sum \deg(v) = 2E$.

W14-M12 — a. A vertex with two neighbours that are taken before it is enqueued by both, and then appears twice in the order (W14-B1). BFS still finds every vertex (b) — it just repeats some.

W14-M13 — b. A stack is **last** in, first out (not a): pushing D then B from A's list B, D puts B on top, so B comes off first — the same choice the recursive for loop makes.

W14-M14 — c. One call frame per vertex on the path; Python's default limit is 1,000 frames. The reference `dfs` already fails on a path of 1,000 edges. `dfs_iterative` handles it.

W14-M15 — d. Directed acyclic graph. (a) describes a tree.

W14-M16 — c. Every vertex keeps in-degree 1, so the ready queue starts empty, nothing is output, and $0 < 3$ vertices means a cycle: None. The `ValueError` (d) is for an **undirected** graph.

W14-M17 — a. When DFS enters B from A, B's list contains A. Without the check every edge would be reported as a cycle (W14-B3).

W14-M18 — c. D was finished when the search backed up from B; it is no longer on the current path $A \rightarrow C$. Only an edge to a **grey** vertex (b) closes a cycle.

W14-M19 — d. {A, B, C}, {D, E}, {F, G}, and H and I on their own. A vertex with no edges is a component of its own; 4 (a) and 3 (b) forget H and I.

W14-M20 — a. In-degree 0 at the start: A and B, queued in node order. A releases nothing (C still waits on B); B releases C and D; C, then D, then E, then F (W14-T5). (b) is also a valid topological order — it is the DFS one — but it is not what Kahn’s algorithm returns. (c) puts C before B, although $B \rightarrow C$.

W14-M21 — c. BFS counts edges; with road lengths the fewest roads is not the fewest kilometres. That needs Dijkstra’s algorithm, which this course only names.

W14-M22 — d. Every vertex is enqueued and dequeued once; every list is read once, $\sum \deg = 2E$ entries in total.

Part B — Short answer and essay

W14-E1 (4)

- **Memory:** list $O(V + E)$ — $V + 2E$ cells undirected; matrix V^2 cells, however few edges.
- **has_edge(u, v):** list $O(\deg u)$, scan u’s neighbours; matrix $O(1)$, one cell. **neighbours(u):** list $O(\deg u)$; matrix $O(V)$, a row scan. **One BFS/DFS:** list $O(V + E)$; matrix $O(V^2)$.
- **Road map,** $V = 100,000$, about three roads per junction (E about 150,000): the list holds about $100,000 + 300,000 = 400,000$ cells; the matrix 10^{10} cells — about 10 GB even at one byte each. The list, without question; a BFS on it is a few hundred thousand steps.
- **500 people, nearly everyone knows everyone:** E is close to $500 \times 499/2 \approx 125,000$ and the matrix is only 250,000 cells — about the same as the list. Each has_edge is $O(1)$ on the matrix against a scan of up to about 500 neighbours on the list; asked millions of times, the matrix wins by a large factor.

W14-E2 (4)

- **Algorithm:** put s in a queue and mark it; repeatedly dequeue a vertex, output it, and enqueue (and mark) each unmarked neighbour. $O(V + E)$.
- **Why shortest:** the queue holds the vertices in non-decreasing order of distance from s — all of distance d before any of distance $d + 1$ — so a vertex is first discovered from a vertex one layer nearer, and no later route can be shorter.
- **Rebuilding the path:** record, for each vertex, the vertex it was discovered from (the parent, None for s). When the goal is dequeued, follow the parents back to s and reverse the list; if the queue empties first, the goal is unreachable — None.
- **Weighted roads:** BFS minimises the **number** of edges. A route of two 40 km roads beats one of three 1 km roads by BFS’s measure. Weighted shortest paths need Dijkstra’s algorithm (a priority queue), beyond this course.

W14-E3 (4)

- **The call stack** holds one frame per vertex on the current path: which vertex it is, and how far its for loop over the neighbours has got. Returning from a call is backing up.
- **Mark on pop:** recursion marks a vertex when it **enters** it, and may reach it by a deeper route before an earlier vertex's loop gets to it. Marking on push would fix it to the earlier route. So allow a vertex to be pushed more than once and skip it if it is already visited when popped.
- **Push in reverse:** the stack returns the last push first; to take the first neighbour first, it must be pushed last.
- **Why bother in Python:** the recursive version fails with `RecursionError` once the path is about 1,000 vertices deep; your own Stack can grow as large as memory allows. (It also shows exactly what recursion was doing.)

W14-E4 (4)

- **Undirected:** DFS remembering each vertex's parent; a visited neighbour that is **not** the parent closes a cycle. The parent must be excluded because every edge is stored at both ends.
- **Directed:** three colours — white (unseen), grey (on the current path), black (finished). An edge to a **grey** vertex closes a cycle; an edge to a black one does not.
- **Parent rule on a directed graph fails:** $A \rightarrow B, B \rightarrow A$ is a cycle, but at B the neighbour A is its parent and is skipped — “no cycle”.
- **Visited-only rule on a directed graph fails:** the diamond $A \rightarrow B \rightarrow D, A \rightarrow C \rightarrow D$, or simply $B \rightarrow C$ and $A \rightarrow C$ searched from B first: C is visited when reached again, and the check says “cycle” wrongly (W14-B4).
- **Colour rule on an undirected graph fails:** at B, entered from A, the neighbour A is grey, so every edge is reported as a cycle.

W14-E5 (3)

- **Definition:** an order of the vertices of a directed graph in which, for every edge $X \rightarrow Y$, X comes before Y. It exists **exactly** when the graph is a DAG: a cycle would need each of its vertices to come before the next.
- **Kahn:** count in-degrees; queue the vertices with in-degree 0; repeatedly dequeue one, output it, and subtract 1 from the in-degree of each vertex it points to, queueing any that reach 0. $O(V + E)$.
- **No order:** if fewer than V vertices are output, the rest wait on each other — they contain a cycle — so return None.
- **Application:** course prerequisites (a valid study plan), build systems (compile after dependencies), package managers, spreadsheet recalculation.

Part C — Trace the algorithms

W14-T1

Take	Adds	Queue afterwards
A	B, D	B D
B	C, E (A is visited)	D C E
D	— (A, E are visited)	C E
C	F	E F
E	— (B, D, F are visited)	F
F	G	G
G	—	empty

Order **A B D C E F G**. Distances from A: A 0; B, D 1; C, E 2; F 3; G 4. E is seen by B first, so when D is taken E is already waiting — that is why D adds nothing.

W14-T2

```
visit A
  visit B                                (A: its first neighbour)
    A already visited
    visit C
      B already visited
      visit F
        C already visited
        visit E
          B already visited
          visit D
            A already visited
            E already visited
            finish D
            F already visited
          finish E
        visit G
          F already visited
          finish G
        finish F
      finish C
      E already visited
    finish B
    D already visited
  finish A
```

Order **A B C F E D G**. The deepest point is 6 frames: A, B, C, F, E, D. G is entered only after D and E have finished — it hangs off F, the first vertex on the way back with an unvisited neighbour.

W14-T3 Stack written bottom → top.

Pop	Action	Push	Stack afterwards
A	visit	D, B	D B
B	visit	E, C	D E C
C	visit	F	D E F
F	visit	G, E	D E G E
E	visit	D	D E G D
D	visit	—	D E G
G	visit	—	D E
E	already visited: skip		D
D	already visited: skip		empty

Order **A B C F E D G** — the same as W14-T2. **D** (by A and by E) and **E** (by B and by F) are pushed twice; in each case the copy pushed later, from deeper in the search, is popped first, exactly as the recursion enters them.

W14-T4 BFS from A records each parent when the vertex is first discovered:

Taken	New parents
A	$B \leftarrow A, D \leftarrow A$
B	$C \leftarrow B, E \leftarrow B$
D	—
C	$F \leftarrow C$
E	—
F	$G \leftarrow F$
G	the goal: stop

Walk back from G: $G \leftarrow F \leftarrow C \leftarrow B \leftarrow A$, reversed: **["A", "B", "C", "F", "G"]**, 4 edges. (A–D–E–F–G is also 4 edges; F's parent is C because C was taken before E.)

`shortest_path_unweighted(H, "D", "C") = ["D", "A", "B", "C"]`. From D the queue is A, E. A is taken first and records $B \leftarrow A$; when E is taken, B already has a parent, so E records only $F \leftarrow E$; then B records $C \leftarrow B$. D–E–B–C and D–E–F–C are also 3 edges long. BFS guarantees a shortest path, and the neighbour order decides which one.

W14-T5 In-degrees at the start: A 0, C 2, B 0, D 1, E 2, F 1. Queue: A B.

Take	In-degrees that change	Reach 0	Queue afterwards
A	C: $2 \rightarrow 1$	—	B
B	C: $1 \rightarrow 0$, D: $1 \rightarrow 0$	C, D	C D
C	E: $2 \rightarrow 1$	—	D
D	E: $1 \rightarrow 0$	E	E
E	F: $1 \rightarrow 0$	F	F
F	—	—	empty

Six vertices out: **A B C D E F. Yes**, B D A C E F is also valid: check every edge — A before C, B before C and D, C and D before E, E before F. It is the order `topological_order_dfs` (W14-C3) returns. A DAG usually has many topological orders.

Part D — Graph state

W14-S1

After	Adjacency lists
<code>add_edge A B</code>	A: B · B: A
<code>add_edge B C</code>	A: B · B: A C · C: B
<code>add_node D</code>	A: B · B: A C · C: B · D: (<i>empty</i>)
<code>add_edge C A</code>	A: B C · B: A C · C: B A · D: (<i>empty</i>)
<code>add_edge D B</code>	A: B C · B: A C D · C: B A · D: B

The matrix at the end (rows and columns A, B, C, D) — symmetric, because the graph is undirected:

	A	B	C	D
A	0	1	1	0
B	1	0	1	1
C	1	1	0	0
D	0	1	0	0

`nodes()` = A B C D. `edges()` = (A, B), (A, C), (B, C), (B, D) — each once; the same list from both classes. `degree("B")` = 3. `add_node D` added a row **and** a column of zeros to the matrix; it would add nothing more if D were added again.

W14-S2

Step	White	Grey (on the path)	Black
enter A	B C D E F	A	
enter B	C D E F	A B	
enter C	D E F	A B C	
finish C (no out-edges)	D E F	A B	C
finish B	D E F	A	B C
enter D (from A)	E F	A D	B C
D → C: C is black — skip	E F	A D	B C
enter E	F	A D E	B C
enter F		A D E F	B C
F → D: D is grey — cycle			

The edge **F** → **D** proves the cycle D → E → F → D: D's call is still running. **D** → **C** reaches a black vertex: C was finished long ago, and nothing leads from C back to D; it is only a

second route to C ($A \rightarrow B \rightarrow C$ and $A \rightarrow D \rightarrow C$). `topological_sort(Q)` returns **None**. Without $F \rightarrow D$ it returns **A B D C E F**.

W14-S3

```
enter A (parent None)
  enter B (parent A)
    A is the parent: skip
  done B
enter C (parent A)
  A is the parent: skip
  enter D (parent C)
    C is the parent: skip
    enter E (parent D)
      D is the parent: skip
      C is visited and not the parent: cycle
```

The cycle C–D–E–C is found at E, looking at its neighbour C: C is visited, and E was entered from D, not from C. Every “skip” line is an edge seen from its other end — without the parent check the search would have stopped, wrongly, at the very first one, B’s neighbour A.

Part E — Complexity analysis

W14-K1. On a Graph: $\Theta(V + E)$ — V iterations of the outer loop, and the inner loops together read every list once, $\sum \deg = 2E$ entries (E for a directed graph). On a Matrix-Graph: $\Theta(V^2)$ — each neighbours call scans a row of V cells, even to return nothing. It computes $\sum_v \deg(v) = 2E$, **twice** the number of edges.

W14-K2. The result has one list per **vertex**, not per component: each component of size k appears k times (in different orders). Cost: every call is a full BFS of that vertex’s component, so a connected graph costs V searches of $O(V + E) = \Theta(V(V + E))$. The correct version starts a search only from an unseen vertex, so every vertex and edge belongs to exactly one search: $\Theta(V + E)$ in total.

W14-K3. On a Graph: for each u , V calls of `has_edge(u, v)`, each $O(\deg u) = \Theta(\sum_u V(1 + \deg u)) = \Theta(V^2 + VE)$; for a dense graph that is $\Theta(V^3)$. On a MatrixGraph: V^2 calls of $O(1) = \Theta(V^2)$. `len(graph.edges())` is $\Theta(V + E)$ on the list and $\Theta(V^2)$ on the matrix: never ask about every pair when you can walk the lists.

Part F — Find and fix the bug

W14-B1. A vertex is marked only when it is dequeued, so while it waits in the queue another vertex can enqueue it again. On `H`, `bfs(H, "A")` returns A B D C E E F F F G G G — E is enqueued by B and by D, F three times, and each copy of F enqueues G again. On the

triangle A-B, A-C, B-C it returns A B C C. `test_bfs_visits_each_node_once` fails. **Fix:** mark the start before the loop, and mark each neighbour **when it is enqueued**:

```
visited.put(start, True)
queue.enqueue(start)
...
    if neighbour not in visited:
        visited.put(neighbour, True)
        queue.enqueue(neighbour)
```

W14-B2. Adding a node that already exists replaces its neighbour list with an empty one — and `add_edge` calls `add_node` on both ends every time. After `add_edge("A", "B")` and `add_edge("B", "C")`, B's list is just [C]: `has_edge("B", "A")` is False while `has_edge("A", "B")` is still True, and `nodes()` is ["A", "B", "B", "C"] (B twice) while `len` is 3. **Fix:** do nothing if the node is already there:

```
def add_node(self, node):
    if node not in self._adjacent:
        self._adjacent.put(node, DynamicArray())
        self._order.append(node)
```

W14-B3. No parent check. In an undirected graph, the vertex you came from is always in your list and always visited, so the function returns True for **any** graph with an edge — even the single edge A-B. **Fix:** pass the parent down and ignore it:

```
def visit(node, parent):
    visited.put(node, True)
    for neighbour in graph.neighbours(node):
        if neighbour not in visited:
            if visit(neighbour, node):
                return True
        elif neighbour != parent:
            return True
    return False
...
    if node not in visited and visit(node, None):
```

W14-B4. “Visited” includes finished vertices, which are not on the current path. Build $B \rightarrow C$ and $A \rightarrow C$, in that order: the loop visits B, then C (finished, no out-edges); then it starts at A, and A's neighbour C is visited — True, though the graph has no cycle. The diamond $A \rightarrow B, A \rightarrow C, B \rightarrow D, C \rightarrow D$ fails the same way (`test_directed_diamond_is_not_a_cycle`). **Fix:** three colours — grey while the call runs, black when it returns; only grey means a cycle:

```
def visit(node):
    colour.put(node, GREY)
    for neighbour in graph.neighbours(node):
        state = colour.get(neighbour, WHITE)
        if state == GREY:
```

```

        return True
    if state == WHITE and visit(neighbour):
        return True
    colour.put(node, BLACK)
    return False

```

Part G — Write the code

The helper constant used by C1 and C5:

```

STEPS = ((-1, 0), (1, 0), (0, -1), (0, 1))          # up, down, left, right

```

W14-C1

```

def maze_distance(maze):
    rows = len(maze)
    cols = len(maze[0]) if rows else 0
    start = None
    for r in range(rows):
        for c in range(cols):
            if maze[r][c] == "S":
                start = (r, c)
    distance = ChainingHashMap()          # cell -> steps from S
    distance.put(start, 0)
    queue = CircularQueue()
    queue.enqueue(start)
    while not queue.is_empty():
        r, c = queue.dequeue()
        if maze[r][c] == "T":
            return distance.get((r, c))
        for dr, dc in STEPS:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc] != "#" \
                and (nr, nc) not in distance:
                distance.put((nr, nc), distance.get((r, c)) + 1)
                queue.enqueue((nr, nc))
    return -1

```

A BFS over the grid, with the cells as vertices named by (row, column) tuples, and the distance map doubling as the visited set. The first time T is dequeued, its distance is the answer; if the queue empties, T cannot be reached. Building a Graph of the cells first, as in the lecture's maze, works too — this version just reads the neighbours straight from the grid. Each cell is enqueued at most once and looks at four neighbours: $O(R \cdot C)$.

W14-C2

```

def is_bipartite(graph):
    colour = ChainingHashMap()                                # vertex -> 0 or 1
    for start in graph.nodes():
        if start in colour:
            continue
        colour.put(start, 0)
        queue = CircularQueue()
        queue.enqueue(start)
        while not queue.is_empty():
            node = queue.dequeue()
            for neighbour in graph.neighbours(node):
                if neighbour not in colour:
                    colour.put(neighbour, 1 - colour.get(node))
                    queue.enqueue(neighbour)
                elif colour.get(neighbour) == colour.get(node):
                    return False                                # an edge inside one colour
    return True

```

Give the start of each component colour 0, and every newly discovered vertex the other colour from the vertex that discovered it. An edge whose two ends end up with the same colour makes two-colouring impossible. **Why an odd cycle fails:** walking round a cycle, the colours must alternate 0, 1, 0, 1, ...; after an odd number of steps you are back at the start with the opposite colour — a contradiction. (The converse also holds: a graph with no odd cycle is bipartite.) The outer loop matters: the odd cycle may be in any component. $O(V + E)$.

W14-C3

```
WHITE, GREY, BLACK = 0, 1, 2
```

```

def topological_order_dfs(graph):
    colour = ChainingHashMap()                                # absent means WHITE
    finished = []

    def visit(node):
        colour.put(node, GREY)
        for neighbour in graph.neighbours(node):
            state = colour.get(neighbour, WHITE)
            if state == GREY:
                return False                                    # back into the current path
            if state == WHITE and not visit(neighbour):
                return False
        colour.put(node, BLACK)
        finished.append(node)                                  # everything after it is done
    return True

```

```

    for node in graph.nodes():
        if colour.get(node, WHITE) == WHITE and not visit(node):
            return None
    finished.reverse()
    return finished

```

A vertex is appended to finished only after every vertex it points to has finished, so in the reversed list it comes before all of them — every edge points forwards. A grey neighbour is an edge back into the current path: a cycle, so no order exists. For P the answer is B D A C E F; for the diamond in the docstring, A C B D. The inner function is recursive, so this version has the same depth limit as dfs.

W14-C4

```

def semester_plan(prerequisites):
    graph = Graph(directed=True)
    for before, after in prerequisites:
        graph.add_edge(before, after)
    waiting = ChainingHashMap() # course -> unmet prerequisites
    for course in graph.nodes():
        waiting.put(course, 0)
    for course in graph.nodes():
        for later in graph.neighbours(course):
            waiting.put(later, waiting.get(later) + 1)
    semester = [course for course in graph.nodes() if waiting.get(course) == 0]
    plan, placed = [], 0
    while semester:
        semester.sort()
        plan.append(semester)
        placed += len(semester)
        following = []
        for course in semester:
            for later in graph.neighbours(course):
                waiting.put(later, waiting.get(later) - 1)
                if waiting.get(later) == 0:
                    following.append(later)
        semester = following
    if placed < len(graph): # the rest wait on each other
        return None
    return plan

```

Kahn's algorithm, but taking a whole layer at a time: the courses whose prerequisites are all done form this semester; finishing them may free the next. **Why the fewest semesters is the longest chain:** along a chain of k courses, each needs the one before, so no plan can finish it in fewer than k semesters. And the layer of a course in this plan is exactly the length of the longest chain ending at it — it is taken the semester after its last prerequisite,

never later. So the plan uses as many semesters as the longest chain, and no plan can use fewer. On the lecture's bylaw graph it gives four. (Real plans are also limited by credit hours per semester, which this ignores.)

W14-C5

```
def count_islands(grid):
    rows = len(grid)
    cols = len(grid[0]) if rows else 0
    seen = ChainingHashMap()
    islands = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != "1" or (r, c) in seen:
                continue
            islands += 1                                # a new island: flood it
            seen.put((r, c), True)
            queue = CircularQueue()
            queue.enqueue((r, c))
            while not queue.is_empty():
                cr, cc = queue.dequeue()
                for dr, dc in STEPS:
                    nr, nc = cr + dr, cc + dc
                    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == "1"
                        and (nr, nc) not in seen:
                            seen.put((nr, nc), True)
                            queue.enqueue((nr, nc))
    return islands
```

Connected components on a grid: every land cell not yet seen starts a new island, and a BFS “floods” it, marking every land cell joined to it. Each cell is seen once: $O(R \cdot C)$. A recursive flood fill is shorter, but an island 2,000 cells long is deeper than Python’s recursion limit — `test_count_islands_on_a_large_grid` checks exactly that.